🤖 autoresearch

by Andrej Karpathy

Complete Layman's Installation & Usage Guide

> 💡 **What is this?** autoresearch lets an AI agent (like Claude) autonomously run ML experiments overnight — it tweaks your neural network code, trains for 5 minutes, checks if the result improved, keeps or discards, and repeats. You wake up to 100+ experiments done while you slept!

SECTION 1 — What You Need Before You Start

Think of this as the **"ingredients list"** before cooking. You need all of these on your computer first.

1.1 Hardware Requirements

> ⚠️ **Important:** This macOS fork supports Apple Silicon Macs (M1/M2/M3/M4 chips) and NVIDIA GPUs. It does NOT run on older Intel Macs or basic CPUs efficiently.

- Apple Silicon Mac (M1, M2, M3, or M4) — any MacBook/Mac Mini/iMac with the M-chip
- OR a machine with a single NVIDIA GPU (e.g. RTX 3080, 4090, A100)
- At least 16 GB of RAM recommended (32 GB+ for best results)
- ~5 GB of free disk space for data and model files

1.2 Software Requirements

Software	What it is / Why needed
macOS	Your operating system (Ventura 13+ recommended)

Python 3.10+	The programming language the project runs in
uv	A fast Python package manager — replaces pip, like npm for JS
Git	Version control — the agent uses it to track experiments
Claude / Codex	The AI agent that will run experiments autonomously

---

SECTION 2 — Installation (Step by Step)

📌 **Note:** All commands below are typed in your Terminal app. On Mac: press Cmd+Space, type "Terminal", press Enter.

Step 1 — Install Homebrew (Mac package manager)

If you don't have Homebrew installed already, open Terminal and paste:

```
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

This is a one-time setup. If it's already installed, you'll see `Homebrew is already installed.`

Step 2 — Install Git

Git lets you manage the project's code versions. In Terminal:

```
brew install git
```

Step 3 — Install Python 3.10 or later

Check if you already have it:

```
python3 --version
```

If it says 3.10 or higher, you're fine. If not, install it:

```
brew install python@3.12
```

Step 4 — Install uv (the package manager this project uses)

uv is what Karpathy chose to manage all Python libraries. Install it with:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

After installation, restart your Terminal or run:

```
source ~/.zshrc
```

Verify it works:

```
uv --version
```

Step 5 — Download the Project

You already have the zip file. Extract it to a folder of your choice — for example, your Desktop:

```
cd ~/Desktop unzip autoresearch-macos-master.zip cd autoresearch-macos-master
```

OR, if you want to clone directly from GitHub:

```
git clone https://github.com/miolini/autoresearch-macos cd autoresearch-macos
```

Step 6 — Install Project Dependencies

Inside the project folder, run:

```
uv sync
```

This reads `pyproject.toml` and automatically installs everything needed — PyTorch, numpy, etc. It may take 2–5 minutes the first time.

Step 7 — Download Training Data (One-Time Setup)

This downloads the dataset and trains a tokenizer. Run:

```
uv run prepare.py
```

This takes **~2 minutes** and only needs to be done once. Data is saved at `~/.cache/autoresearch/`

Step 8 — Test That Everything Works

Run a single 5-minute training experiment to confirm setup:

```
uv run train.py
```

After ~5 minutes, you should see output like:

```
--- val_bpb:          0.997900 training_seconds: 300.1 total_seconds:    325.9
peak_vram_mb:     45060.2 mfu_percent:      39.80 total_tokens_M:   499.6
num_steps:        953 num_params_M:     50.3
```

✅ **Success!** If you see "val_bpb" in the output, your setup is complete and working perfectly. Lower val_bpb = better model.

SECTION 3 — Understanding the 3 Key Files

The whole project has just 3 files you need to understand:

File	Role & Rules
prepare.py	Fixed utility file — downloads data, trains tokenizer, runs evaluation. NEVER modify this.
train.py	The AI agent's playground — model architecture, optimizer, training loop. Agent modifies ONLY this.
program.md	Your instructions to the AI agent — what goals to pursue, constraints, how to log results. YOU edit this.

🔑 **Key Concept:** The metric is val_bpb (validation bits-per-byte). Lower = better. Every experiment runs for exactly 5 minutes, so results are fairly comparable regardless of what changes the agent makes.

SECTION 4 — Running Autonomous Research (The Main Event)

4.1 The Big Picture

Once everything is set up, you point Claude (or another coding agent) at this folder, give it the prompt below, and let it run autonomously — ideally overnight.

💬 **Starter Prompt to give Claude:** Hi! Please have a look at program.md in this repo and let's kick off a new experiment. Do the full setup first, then start the experiment loop and keep running until I stop you.

4.2 What the Agent Does in Each Experiment Loop

1. Reads the current state of the branch and train.py
2. Comes up with a hypothesis (e.g. 'what if I increase the learning rate?')
3. Edits train.py with the change
4. Commits the change with git
5. Runs: uv run train.py > run.log 2>&1
6. Reads the result: grep val_bpb run.log
7. If improved → keeps the commit (advances the branch)

8. If worse → reverts with git reset
9. Logs the result to results.tsv
10. REPEATS — forever until you manually stop it

4.3 Setting Up the Git Branch (Agent Does This)

The agent will set up a dated branch for you. But if you want to do it manually:

```
git checkout -b autoresearch/mar10 touch results.tsv echo -e
'commit\tval_bpb\tmemory_gb\tstatus\tdescription' > results.tsv
```

SECTION 5 — Customizing program.md for Your Project

This is where **you** have control. The `program.md` file is your 'research org bible' — it tells the AI agent what to focus on.

5.1 What You Can Customize

- **Swap the default dataset for your own text data (e.g. financial reports, domain documents)** Dataset:
- **Change the objective — e.g. 'optimise for speed' or 'try transformer-free architectures'** Research goal:
- **Add rules like 'never exceed 6 GB VRAM' or 'only try changes with known research precedent'** Constraints:
- **Add domain expertise context to guide the agent's hypotheses** Agent persona:

5.2 Example Customisation for a Finance Project

Add to program.md:

```
## Domain Context This is a language model trained on Indian financial regulatory
documents (RBI circulars, SEBI guidelines, AMC factsheets). The goal is to
minimize val_bpb on this specific domain corpus, not general text. Prefer smaller,
fast architectures that could run on a single V100.  ## Constraints - Max VRAM: 20
GB - Prefer architectures with fewer than 100M parameters - Do not try mixture-of-
experts — too complex for this setup
```

SECTION 6 — Reading Your Results

6.1 The results.tsv File

After experiments run, check your log:

```
cat results.tsv
```

You'll see something like:

```
commit    val_bpb      memory_gb    status description a1b2c3d      0.997900
   44.0   keep   baseline b2c3d4e   0.993200     44.2   keep   increase LR to 0.04
c3d4e5f   1.005000     44.0   discard      switch to GeLU activation d4e5f6g
   0.000000     0.0    crash  double model width (OOM)
```

6.2 How to Interpret Results

Column	Meaning
commit	The 7-character git hash of that experiment's code
val_bpb	Validation bits-per-byte — the key metric. LOWER is better.
memory_gb	Peak GPU memory used. Watch this doesn't grow uncontrollably.
status	keep = improvement kept | discard = reverted | crash = experiment failed
description	What the agent tried in plain English

6.3 Expected Output Scale

- ~12 experiments per hour (each experiment = 5 min training)
- ~100 experiments while you sleep (8 hours)
- Each improvement is permanently kept on the branch
- You can see the evolution with: git log --oneline

SECTION 7 — Troubleshooting Common Issues

Problem	Solution
uv: command not found	Restart Terminal after installing uv, or run: source ~/.zshrc
prepare.py fails with download error	Check internet connection. The script downloads ~1 GB of data.
train.py crashes with OOM	Normal — agent will log as 'crash' and move on. Not a setup issue.
val_bpb not printed	The run crashed. Check: tail -n 50 run.log for the Python error.
Agent stops asking questions	Remind it: 'Do NOT stop — keep experimenting until I tell you to stop.'

MPS / Metal errors on Mac	This fork is specifically patched for MPS. Ensure you're using the macos fork, not the original repo.
Git not initialised error	Run: git init && git add . && git commit -m 'initial'

SECTION 8 — Quick Reference Card

Task	Command
Install dependencies	uv sync
Download data (once)	uv run prepare.py
Run one experiment	uv run train.py
Run experiment to log file	uv run train.py > run.log 2>&1
Check result	grep 'val_bpb' run.log
Check for crashes	tail -n 50 run.log
See all experiments	cat results.tsv
See git history	git log --oneline
Stop the agent	Press Ctrl+C in Terminal
Start fresh branch	git checkout -b autoresearch/mar10

🚀 **You're ready!** Setup → uv sync → uv run prepare.py → uv run train.py (test) → Point Claude at the folder with the starter prompt → Go to sleep → Wake up to 100 experiments!

Output :-

```
Vocab size: 8,192
Model config: {'sequence_len': 2048, 'vocab_size': 8192, 'n_layer': 4,
'n_head': 2, 'n_kv_head': 2, 'n_embd': 256, 'window_pattern': 'L'}
Parameter counts:
  wte                     : 2,097,152
  value_embeds            : 4,194,304
  lm_head                 : 2,097,152
  transformer_matrices    : 3,145,856
  scalars                 : 8
  total                   : 11,534,472
Estimated FLOPs per token: 5.662387e+07
Scaling AdamW LRs by 1/sqrt(256/768) = 1.732051
```

```
Time budget: 300s
Gradient accumulation steps: 2
step 00322 (100.0%) | loss: 4.706068 | lrm: 0.00 | dt: 1039ms | tok/sec:
7,887 | mfu: 0.0% | epoch: 1 | remaining: 0s
---
val_bpb:          1.696474
training_seconds: 301.0
total_seconds:    1184.9
peak_vram_mb:     0.0
mfu_percent:      0.05
total_tokens_M:   2.6
num_steps:        323
num_params_M:     11.5

depth:            4
```

autoresearch-macos by Andrej Karpathy · MIT License